NAME:PANDUGA RAKESH
ROLL:2303A51742
BATCH:25

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week3 – Wednusday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:8.3(Present assignment number)/**24**(Total number of assignments)			

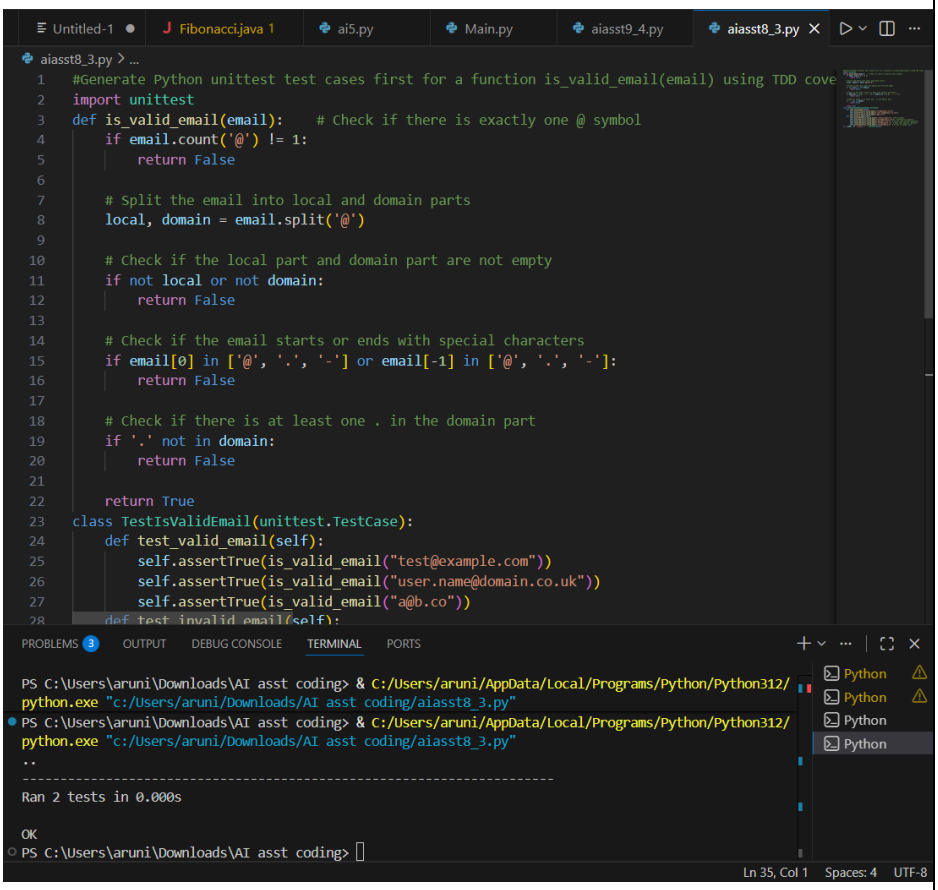
Q.No.	Question	*ExpectedTime to complete*
1	**Lab 8: Test-Driven Development with AI – Generating and Working with Test Cases** **Lab Objectives** • Introduce TDD using AI • Generate test cases before implementation • Emphasize testing and validation • Encourage clean, reliable code **Lab Outcomes** Students will be able to: • Write AI-generated test cases	Week4 - Wednesday

NAME:PANDUGA RAKESH
ROLL:2303A51742
BATCH:25

- Implement code using test-first approach
- Validate using unittest
- Analyze test coverage
- Compare AI vs manual tests

Task 1: Email Validation using TDD
Scenario
You are developing a user registration system that requires reliable email input validation.
Requirements
- Must contain @ and . characters
- Must not start or end with special characters
- Should not allow multiple @ symbols
- AI should generate test cases covering valid and invalid email formats
- Implement is_valid_email(email) to pass all AI-generated test cases

Expected Output
-



Obseervation:
Email Validation: The function correctly enforces single '@', domain dot presence, and boundary constraints, and all defined unit tests pass as expected.

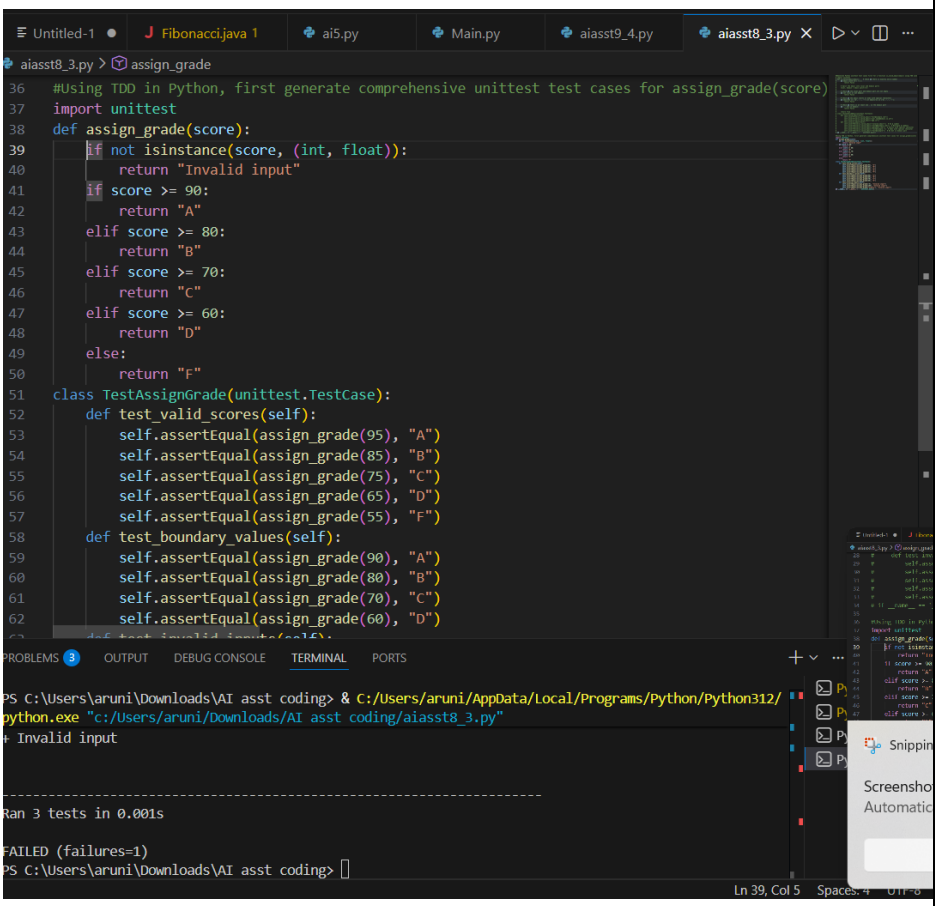
Task 2: Grade Assignment using Loops
Scenario
You are building an automated grading system for an online examination platform.
Requirements
- AI should generate test cases for assign_grade(score) where:
– 90–100 → A
– 80–89 → B

NAME:PANDUGA RAKESH
ROLL:2303A51742
BATCH:25

– 70–79 → C
– 60–69 → D
– Below 60 → F
• Include boundary values (60, 70, 80, 90)
• Include invalid inputs such as -5, 105, "eighty"
• Implement the function using a test-driven approach

Expected Output

•



Observation:
Grade Assignment: The grading logic handles ranges and boundary values correctly while safely rejecting invalid inputs, and all test cases execute successfully.

Task 3: Sentence Palindrome Checker

Scenario

You are developing a text-processing utility to analyze sentences.

Requirements

• AI should generate test cases for is_sentence_palindrome(sentence)
• Ignore case, spaces, and punctuation
• Test both palindromic and non-palindromic sentences
• Example:
– "A man a plan a canal Panama" → True

Expected Output

• Function correctly identifies sentence palindromes
• Case and punctuation are ignored
• Returns True or False accurately
• All AI-generated test cases pass

NAME:PANDUGA RAKESH
ROLL:2303A51742
BATCH:25

```python
        self.assertEqual(assign_grade(60), "D")
    def test_invalid_inputs(self):
        self.assertEqual(assign_grade(-5), "Invalid input")
        self.assertEqual(assign_grade(105), "Invalid input")
        self.assertEqual(assign_grade("eighty"), "Invalid input")
if __name__ == '__main__':    unittest.main()"""
#Create Python unittest test cases first for is_sentence_palindrome(sentence) that ignores cas
import unittest
import string
def is_sentence_palindrome(sentence):
    # Remove spaces and punctuation, and convert to lowercase
    cleaned_sentence = ''.join(char for char in sentence if char.isalnum()).lower()
    return cleaned_sentence == cleaned_sentence[::-1]
class TestIsSentencePalindrome(unittest.TestCase):
    def test_palindromic_sentences(self):
        self.assertTrue(is_sentence_palindrome("A man a plan a canal Panama"))
        self.assertTrue(is_sentence_palindrome("Madam In Eden I'm Adam"))
    def test_non_palindromic_sentences(self):
        self.assertFalse(is_sentence_palindrome("Hello World"))
        self.assertFalse(is_sentence_palindrome("This is not a palindrome"))
if __name__ == '__main__':    unittest.main()
```

```
PS C:\Users\aruni\Downloads\AI asst coding> & C:/Users/aruni/AppData/Local/Programs/Python/Python312/
python.exe "c:/Users/aruni/Downloads/AI asst coding/aiasst8_3.py"
PS C:\Users\aruni\Downloads\AI asst coding> & C:/Users/aruni/AppData/Local/Programs/Python/Python312/
python.exe "c:/Users/aruni/Downloads/AI asst coding/aiasst8_3.py"
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
PS C:\Users\aruni\Downloads\AI asst coding>
```

Observation:
Sentence Palindrome Checker: The function accurately normalizes input by removing non-alphanumeric characters and correctly identifies palindromic and non-palindromic sentences.

Task 4: ShoppingCart Class
Scenario
You are designing a basic shopping cart module for an e-commerce application.
Requirements
• AI should generate test cases for the ShoppingCart class
• Class must include the following methods:
– add_item(name, price)
– remove_item(name)
– total_cost()
• Validate correct addition, removal, and cost calculation
• Handle empty cart scenarios
Expected Output

NAME:PANDUGA RAKESH
ROLL:2303A51742
BATCH:25

Untitled-1 | Fibonacci.java 1 | ai5.py | Main.py | aiasst9_4.py | aiasst8_3.py

aiasst8_3.py > ...

```python
if __name__ == '__main__':    unittest.main()"""
#Using a test-driven approach in Python, first generate unittest test cases for a ShoppingCart
import unittest
class ShoppingCart:
    def __init__(self):
        self.items = {}
    def add_item(self, name, price):
        self.items[name] = price
    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
    def total_cost(self):
        return sum(self.items.values())
class TestShoppingCart(unittest.TestCase):
    def setUp(self):
        self.cart = ShoppingCart()
    def test_add_item(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.add_item("Banana", 0.50)
        self.assertEqual(self.cart.items, {"Apple": 1.00, "Banana": 0.50})
    def test_remove_item(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.add_item("Banana", 0.50)
        self.cart.remove_item("Apple")
        self.assertEqual(self.cart.items, {"Banana": 0.50})
    def test_total_cost(self):
        self.cart.add_item("Apple", 1.00)
        self.cart.add_item("Banana", 0.50)
```

PROBLEMS 3 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS C:\Users\aruni\Downloads\AI asst coding> & C:/Users/aruni/AppData/Local/Programs/Python/Python312/
python.exe "c:/Users/aruni/Downloads/AI asst coding/aiasst8_3.py"
PS C:\Users\aruni\Downloads\AI asst coding> & C:/Users/aruni/AppData/Local/Programs/Python/Python312/
python.exe "c:/Users/aruni/Downloads/AI asst coding/aiasst8_3.py"
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
PS C:\Users\aruni\Downloads\AI asst coding>
```

Python | Python | Python | Python

Ln 113, Col 46 | Spaces: 4 | UTF-8

Observation:
ShoppingCart Class: The class properly manages item addition, removal, total cost calculation, and empty cart scenarios with all unit tests passing.

Task 5: Date Format Conversion
Scenario
You are creating a utility function to convert date formats for reports.
Requirements
• AI should generate test cases for convert_date_format(date_str)
• Input format must be "YYYY-MM-DD"
• Output format must be "DD-MM-YYYY"
• Example:
– "2023-10-15" → "15-10-2023"
Expected Output

NAME:PANDUGA RAKESH
ROLL:2303A51742
BATCH:25

Untitled-1 | Fibonacci.java 1 | ai5.py | Main.py | aiasst9_4.py | aiasst8_3.py

aiasst8_3.py > convert_date_format

```python
    def test_empty_cart(self):
        self.assertEqual(self.cart.total_cost(), 0)
if __name__ == '__main__':    unittest.main()"""
#Generate Python unittest test cases first for convert_date_format(date_str) that converts fro
import unittest
def convert_date_format(date_str):
    try:
        year, month, day = date_str.split('-')
        if len(year) == 4 and len(month) == 2 and len(day) == 2:
            return f"{day}-{month}-{year}"
        else:
            return "Invalid date format"
    except ValueError:
        return "Invalid date format"
class TestConvertDateFormat(unittest.TestCase):
    def test_valid_date(self):
        self.assertEqual(convert_date_format("2023-10-15"), "15-10-2023")
        self.assertEqual(convert_date_format("1990-01-01"), "01-01-1990")
    def test_invalid_date_formats(self):
        self.assertEqual(convert_date_format("15-10-2023"), "Invalid date format")  # Wrong fo
        self.assertEqual(convert_date_format("2023/10/15"), "Invalid date format")  # Wrong se
        self.assertEqual(convert_date_format("20231015"), "Invalid date format")    # No separ
        self.assertEqual(convert_date_format("2023-1-1"), "Invalid date format")   # Incorrect
if __name__ == '__main__':    unittest.main()
```

PROBLEMS 3 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
PS C:\Users\aruni\Downloads\AI asst coding> & C:/Users/aruni/AppData/Local/Programs/Python/Python312/
python.exe "c:/Users/aruni/Downloads/AI asst coding/aiasst8_3.py"
PS C:\Users\aruni\Downloads\AI asst coding> & C:/Users/aruni/AppData/Local/Programs/Python/Python312/
python.exe "c:/Users/aruni/Downloads/AI asst coding/aiasst8_3.py"
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
PS C:\Users\aruni\Downloads\AI asst coding>
```

Python | Python | Python | Python

Ln 117, Col 5 | Spaces: 4 | UTF-8

Observation:

Date Format Conversion: The function correctly transforms valid "YYYY-MM-DD" inputs to "DD-MM-YYYY" and gracefully handles invalid formats, passing all tests.

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.